

PERSONAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF AERONAUTICS

Comparative Study of ML Methods for Aero-Engine RUL Prediction

Author:

Lea Liu (CID: 02561129)

Date: September 18, 2026

1 Introduction

The main purpose of this research is to compare the accuracy and computational cost of different methods for the prediction of the RUL of aircraft engines. Based on the original C-MAPSS data provided by the NASA Open Data Portal ?.

This research compared the applicstion of different metohd to train a RUL predict model to the aero engine using same group of data. The compared methology includes the Linear Regression, regularised linear model, random Forest, XGBoost, CNN and LSTM. Later, the comparision between different methods have been conducted and covers the range of RMSE, MAE, R^2 and traing time.

The optimal method was found by balace the trade off between the error (mainly considered ther MAE) and the training time.

2 Exploratory Data Analysis

To all collected data, there are 4 groups of data from FD001 to FD004, which represent the four different versions of the same simulated-engine benchmark. They differ mainly in operating conditions and fault modes.

	Fault modes	
	1	2
Conditions 1	FD001 Easy	FD003 More faults
Conditions 6	FD002 More varied	FD004 Hardest

The FD001 and FD002 datasets only have one fault model, which is HPC degradation. The FD003 and FD004 datasets, the 2 fault models, correspond to HPC degradation and fan degradation, respectively.

Consequently, the dataset contains two main variables: engine fault mode and operating conditions. The fault mode captures issues occurring either in the High Pressure Compressor (HPC) or the fan. The operating conditions consist of either a single baseline condition (sea level only) or a set of six distinct operating conditions, which make the model built more complicated.

3 Method A: Linear Regression

The linear regression method has been applied as the baseline method to compare with the rest of the advanced methods.

4 Method B: Regularised linear model (Ridge)

The data sweep has been processed from 10^{-3} to 10^3 in the Alpha range, spent 0.1336 seconds and found the minimum MAE error of 25.96 at Alpha equal to 0.291.

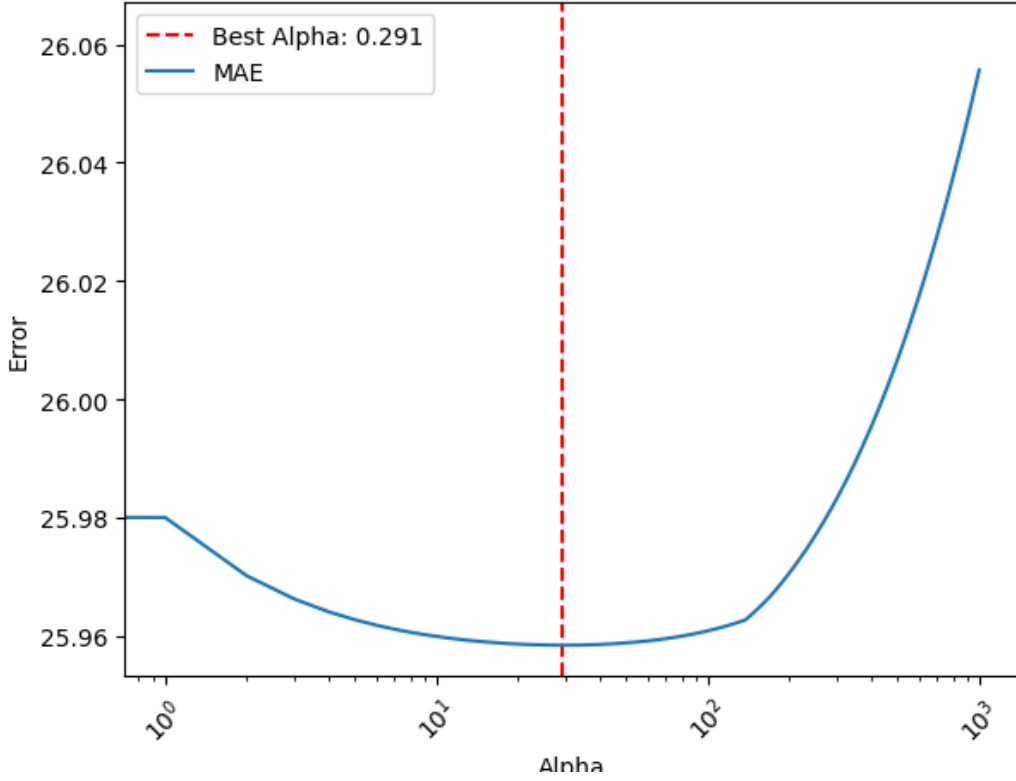


Figure 1: Ridge Model Tuning: Optimal $\alpha = 0.291$

Compared with the baseline, the Ridge model achieves a 0.95% lower MAE and 6.8 times faster training time.

As the non-regularised linear model $\alpha = 0$ and the ridge model with the $\alpha = 0.291$ which is a mild shrinkage.

The rigid linear regression model is defined to find the minimised value of the following equations:

$$\min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2 \quad (1)$$

The X are the features of the aero engine, the y are the actual test scores, and the β is the weight given to each feature. Hence, the first term is the residual sum of squares, which measures how well the model fits the data, and the second term is the penalty term (L2 regularisation), which measures the squared magnitude of the coefficients to avoid overfitting.

The change from 0 to 0.291 emphasises the penalty more than the baseline and makes the new model gain a smoother and more reliable result within the loss of tiny accuracy.

5 Method C: Random Forest

The random forest model is not a perfect fit for this type of machine learning model building. The result of the random forest is MAE equals to 19.85, but with the training time of 2270 seconds.

To investigate the deep reason behind this long training time, we have to consider the own property ensemble learning model, which is formed by multiple decision trees. Each tree is trained independently on a bootstrap sample of the data, and at each node, only a random subset of features is considered for splitting.

As a result, the total training time scales linearly with the number of trees and logarithmically with the sample size and feature dimension, making it significantly more computationally expensive than simpler models like linear regression or ridge regression. Hence, the random forest model is not favourable to the aero engine data analysis.

For more detail, the random forest model tested the following hyperparameters during the grid search, which have been shown in Table 1.

Table 1: Random Forest hyperparameter search space

Hyperparameter	Values
n_estimators	100, 200, 300 , 500
max_depth	10 , 15, 20, 30, None
min_samples_split	2, 5, 10
min_samples_leaf	1 , 2, 4
max_features	0.5 , 0.7, 1.0

The grid search optimization identified an optimal model configuration that balances complexity with robust regularisation.

The best-performing model used 300 estimators and capped the maximum tree depth at 10 to mitigate overfitting while maintaining sufficient representational capacity. The minimum samples split was set to 10, applying a conservative threshold that demands strong statistical evidence before splitting a node, thereby reducing variance and improving generalisation. By comparison, the minimum sample leaf was kept at 1, enabling more granular leaf nodes when supported by the data.

Furthermore, the model limited the number of features considered at each split to 50%. This limitation encourages reduced correlation across trees by diversifying the feature subsets assessed at each node, thereby strengthening the ensemble and lowering the risk of overfitting in high-dimensional spaces.

To sum up, these hyperparameters reflect a deliberate trade-off between model flexibility and stability, resulting in a setup that is both streamlined and empirically effective.

6 Method D: XGBoost Regression

The engine degradation is a non-linear behaviour. It runs smoothly in early life with minimal wear and gradually turns to linear degradation in mid-life. Later, in late life, the engine with rapid degradation due to the rapid wear.

Hence, the non-linear XGBoost regression model fits the engine URL prediction well and better than the above linear models, as it is able to create different rules for diffednt cycle ranges.

Compared to the Random Forest model, which is good but only averages trees trained by randomly chosen samples. XGBoost is intelligent and more focused on hard cases to be able to make more advanced and precise predictions.

In order to predict the URL of the engine of the aircraft, which is crucial for the safety of all travelling passengers and aircraft, the trained models have adopted the biased penalty parameter as shown in Fig. 2.

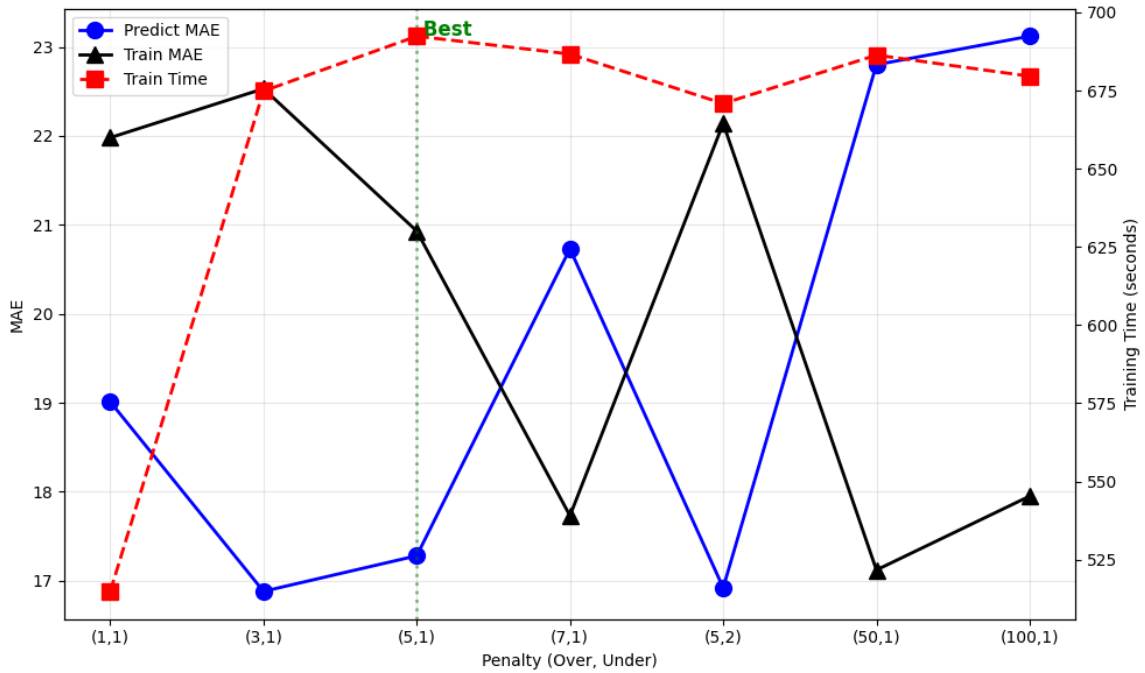


Figure 2: XGBoost Comparision

Configuration 3 sacrifices only 0.4 predict MAE compared to the best predict MAE, with the value of 17.28. However, the overcount of the configuration 3 has been reduced by 17% to 44%. To the higher overpenalty values, the consumption of MAE to overcount is too less efficient.

Moreover, all training time falls in the range of 670 - 692 seconds, so the extra 20s than configuration 2 (3,1) is negligible for the safety gained.

Hence, the optimal combination of penalty parameters was found from the trade-off between the predicted MAE and overestimation, where:

$$(Over\ penalty, Under\ penalty) = (5, 1) \quad (2)$$

7 Method E: Convolutional Neural Network (CNN)

Compared to the above models, the convolutional neural network machine learning models showed much better results on the MAE.

As shown in Fig. 3, the optimal result can be found at the left lower corner, which has the over/under estimation ratio 3:1. The best OOF MAE at the point 3:1 is 14.0043. Although the 1:1 model was 24 seconds ($\sim 4.68\%$) faster than the 3:1 model, the 5% overestimation still undermines the safety considerations for the aircraft engine URL assessment.

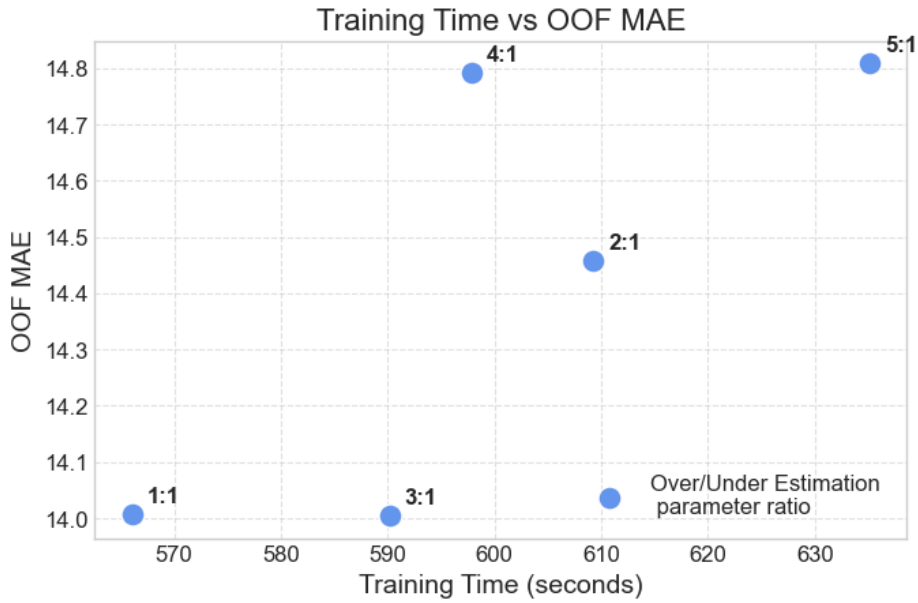


Figure 3: Training time OOF MAE Trade-off

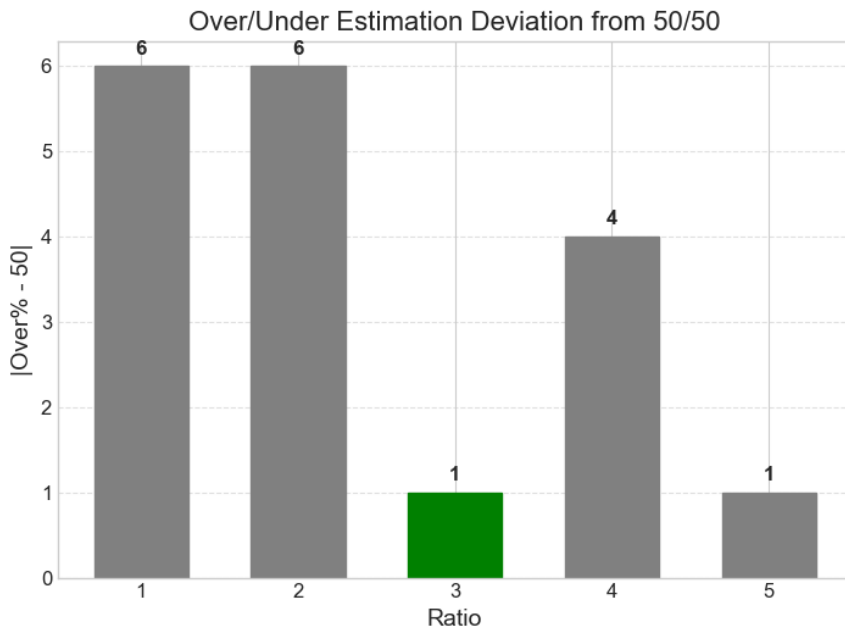


Figure 4: Over/underestimation deviation comparison

Moreover, the hyperparameter optimisation was conducted using a search space covering the CNN architecture, regularisation, learning rate, batch size, and weight decay. The final CNN model was evaluated using a 10-fold GroupKFold ensemble. The selected architecture consisted of three convolutional layers with channel dimensions of (128, 64, 32), a kernel size of 3, and a dropout rate of 0.1. The learning rate was set to 1×10^{-3} , with a batch size of 128 and no weight decay.

The selected configuration used the largest initial channel dimension among the candidate architectures, followed by progressively smaller channel dimensions. This architecture may allow the network to extract a relatively broad set of initial features while reducing the representation dimensionality in subsequent layers.

The selected dropout rate of 0.1 corresponds to relatively weak dropout regularisation among the tested options. Similarly, the selected weight decay of zero indicates that explicit L2-style weight regularisation was not used through this hyperparameter. These choices should be interpreted in conjunction with the observed training and validation errors, rather than as evidence that stronger regularisation would necessarily reduce predictive performance.

Table 2: Hyperparameter search space and selected configuration.

Hyperparameter	Search space
Channels	(32, 64, 32), (64, 128, 64), (64, 64, 64), (128, 64, 32)
Kernel size	3, 5
Dropout	0.1, 0.3, 0.5
Learning rate	10^{-3} , 5×10^{-4} , 10^{-4}
Batch size	128, 256, 512
Weight decay	0, 10^{-5} , 10^{-4}

The mean validation MAE across the ten folds was 14.0053, which closely agrees with the overall OOF result of 14.0043. This consistency suggests that the reported OOF performance is representative of the aggregate cross-validation results.

The final ten-fold training procedure required 590.16 seconds (9.84 min). In contrast, prediction required only 0.0632 seconds for the evaluated prediction workload.

Table 3: Final CNN model performance.

Metric	Result
Prediction MAE	14.4098
OOF MAE	14.0043
Mean fold validation MAE	14.0053
Training MAE	3.6411
RMSE	18.9261
R^2	0.7926
Training time (10 folds)	590.16 s
Prediction time	0.0632 s

The GroupKFold prevents samples from a given group from being distributed across both training and validation sets and provides an assessment of generalisation to held-out groups, rather than merely to randomly held-out individual observations. The validation MAE of each group has been summarised in Table 4 and reflects the differences in the distribution, complexity, or difficulty of the groups assigned to each validation fold.

Table 4: Validation MAE for each GroupKFold fold.

Fold	Validation MAE
1	15.7056
2	16.1290
3	12.0729
4	15.4722
5	12.4119
6	10.0764
7	17.8745
8	11.8936
9	15.6724
10	12.7450
Mean	14.0053

The prediction errors were further examined to assess the direction of the model’s deviations.

Table 5: Overestimation and underestimation analysis for 100 evaluated predictions.

Error type	Count	Mean magnitude
Overestimation	51 (51%)	15.8643
Underestimation	49 (49%)	12.8960

The model overestimated the target in 51% of the evaluated cases and underestimated it in 49%. The near-equal frequency of the two error directions suggests that the predictions were not strongly biased towards either overestimation or underestimation in terms of occurrence.

8 Summary

This project compares machine learning methods for predicting the Remaining Useful Life (RUL) of aircraft engines using NASA C-MAPSS data (FD001—FD004), which vary in operating conditions and fault modes (HPC and fan degradation). Six approaches were evaluated: linear regression, Ridge regression, random forest, XGBoost and CNN.

Linear regression served as the baseline. Ridge regression ($\alpha = 0.291$) achieved a 0.95% lower MAE (25.96) and trained 6.8× faster than the baseline.

Random Forest reached MAE 19.85 but required 2270 seconds, making it impractical.

XGBoost captured non-linear degradation well, with optimal over/under-penalty (5, 1) yielding predicted MAE ≈ 17.28 while reducing overestimation by 17%–44%.

The CNN performed best, achieving OOF MAE 14.0043 and 590.16 s training time via 10-fold GroupKFold. Its errors were nearly balanced (51% overestimation, 49% underestimation).

Overall, the CNN offered the best accuracy-cost trade-off.